

**“Intelligent Water Distribution Monitoring and
Leakage Detection Platform”**

CO4 ASSESSMENT TOOL 2

Submitted in partial fulfilment for the Course of

CSA10 / CSA1063 – Software Engineering

to the award of the degree of

BACHELOR OF ENGINEERING

IN

Computer Science and Engineering

Submitted by

Lenin K – [192321050]

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

2026

DECLARATION

I, Lenin K, Reg. No. [192321050], of the Department of Computer Science and Engineering, Saveetha Institute of Medical and Technical Sciences, Saveetha University, Chennai, hereby declare that the Capstone/Assessment work entitled **“Intelligent Water Distribution Monitoring and Leakage Detection Platform”** is the result of my own bonafide effort. To the best of my knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics.

Place: Chennai

Date: 18/08/2026

Signature of the Student

Document Scope

This design document is organized to mirror the evaluation rubric for the CI/CD Pipeline Design Exercise. Sections 1 and 2 analyze the assigned problem statement and design the pipeline architecture and workflow. Section 3 selects and justifies the tools and technologies used at every stage. Section 4 defines the automated testing and quality-integration strategy. Section 5 details the deployment environments, embedded security practices, and rollback mechanism. Section 6 walks through a concrete, end-to-end pipeline execution with supporting diagrams. Sections 7 and 8 assess platform-specific risks and the ethical/compliance considerations relevant to a critical-infrastructure monitoring product, and Section 9 concludes the document.

List of Figures

- Figure 1. End-to-End CI/CD Pipeline Architecture
- Figure 2. Git Branching Strategy (GitFlow) and Pipeline Triggers
- Figure 3. Sample GitHub Actions Workflow Configuration (YAML)
- Figure 4. Multi-Environment Promotion Path with Blue-Green Deployment

List of Tables

- Table 1. Tools & Technology Selection with Justification
- Table 2. Pipeline Quality Gate Pass Criteria
- Table 3. Requirement-to-Pipeline-Feature Traceability
- Table 4. Risk Assessment & Mitigation Matrix
- Table 5. Roles & Responsibilities (RACI)

1. Introduction & Requirement Analysis

1.1 Project Overview

The assigned problem statement is an Intelligent Water Distribution Monitoring and Leakage Detection Platform — an IoT-driven system that continuously monitors a municipal or industrial water-distribution network using flow, pressure, and acoustic sensors deployed across pipelines, junctions, and reservoirs. The platform ingests high-frequency telemetry from thousands of field sensors, applies statistical and machine-learning-based anomaly detection to flag probable leaks or pipe bursts, localizes the likely leak point on a GIS-based network map, and pushes actionable alerts to control-room operators and field technicians. Because the system is safety-critical, data-intensive, and continuously streaming, its CI/CD pipeline must be designed around reliable high-throughput ingestion, rigorous validation of the leak-detection model before every release, and near-zero downtime for the monitoring and alerting path.

1.2 Core System Features Driving CI/CD Requirements

- **IoT telemetry ingestion:** Continuous ingestion of flow, pressure, acoustic, and water-quality readings from field sensors and gateways over MQTT, buffered through a streaming layer before persistence.
- **Leak-detection engine:** A combination of rule-based thresholds and a trained ML anomaly-detection model that scores incoming telemetry in near real time and flags probable leaks, bursts, or non-revenue-water loss.
- **GIS-based network visualization:** An interactive map of the pipeline network showing live sensor status, estimated leak location, and historical incident overlays.
- **Alerting & work-order workflow:** Automatic SMS/push/email alerts to control-room operators and field crews, with role-based ticket assignment, acknowledgement, and closure tracking.
- **Predictive maintenance & analytics:** Historical trend dashboards and predictive scoring that flag pipe segments at elevated risk of future failure based on age, pressure history, and past incidents.
- **Account & access management:** Role-based accounts for Utility Admins, Control-Room Operators, Field Technicians, and Auditors, plus integration hooks into existing SCADA and municipal asset-management systems.

1.3 Identified CI/CD Needs

Based on the above feature set, the following CI/CD requirements were identified for the platform:

- **1. Frequent, low-risk integration:** Continuous Integration on every commit and pull request validates changes to the ingestion service and leak-scoring logic immediately, since a regression here can silently blind operators to a real leak.
- **2. Rigorous validation of the leak-detection model:** The anomaly-detection/ML model and its feature pipeline must be covered by automated accuracy tests (precision, recall, false-positive rate against a labelled leak-event dataset) before any promotion, as both missed leaks and alert fatigue from false positives carry real-world cost.
- **3. Near-zero downtime deployment:** Because the platform monitors live infrastructure around the clock, the pipeline must support zero-downtime releases (blue-green with canary traffic) so the monitoring and alerting path is never silently unavailable during a deployment.

- **4. Security and critical-infrastructure compliance:** As a system ingesting data from field devices and interfacing with utility SCADA systems, the pipeline must integrate static/dynamic security scanning, dependency vulnerability checks, and device-authentication safeguards at every build.
- **5. Scalability validation:** The system must scale to tens of thousands of concurrent sensor streams; pipeline-integrated load testing verifies ingestion throughput and alerting latency before every production release.
- **6. Fast, safe rollback:** Any failed deployment, or any release that causes an anomalous spike or drop in leak alerts, must be detected via health checks and automatically rolled back to protect continuous monitoring coverage.

1.4 Stakeholders & Environments

The pipeline serves three primary stakeholder groups — the development team (frequent commits to feature branches for dashboard, ingestion, and model code), the QA/data-science review team (validating builds and model accuracy on staging before sign-off), and utility operators and field technicians (who rely exclusively on thoroughly tested production releases for real-time leak alerts). Three corresponding environments are used: Development (continuous auto-deploy for internal testing), Staging (production-like, used for QA sign-off, model shadow-testing, and security scanning), and Production (blue-green with canary rollout, operator-facing, high availability).

1.5 Assumptions & Constraints

- The platform's dashboard/GIS front end is built in React/TypeScript, backend ingestion and API services in Node.js, and the leak-detection model service in Python; services are packaged as Docker containers and orchestrated on Kubernetes — this drives the npm/pip/Jest/PyTest/Docker-centric tooling used throughout the pipeline.
- The team operates in short, frequent sprints with multiple daily commits, so the pipeline is optimized for fast feedback (parallelized test stages, incremental builds) rather than long, infrequent batch releases.
- The product is hosted on a public cloud provider (assumed AWS) with managed Kubernetes (EKS), a managed IoT/MQTT ingestion endpoint (AWS IoT Core), and a container registry (ECR); an equivalent pipeline could be adapted to Azure/GCP with minimal changes to the deployment stage only.
- Because the platform monitors physical infrastructure and informs safety-relevant field dispatch, reliability, data integrity, and the accuracy of the leak-detection model are treated as first-class, non-functional requirements throughout the design — not an afterthought bolted on at release time.
- Budget and infrastructure constraints assume a small-to-mid-size engineering team (5–15 developers, including one data scientist), which favors managed/SaaS tooling (GitHub Actions, SonarCloud/SonarQube, Snyk, MLflow) over self-hosted alternatives that would need dedicated DevOps headcount.

2. Pipeline Architecture & Workflow

2.1 High-Level Pipeline Design

The pipeline is triggered automatically on every code push and pull-request merge and executes seven sequential stages — Source Code Management, Build, Automated Testing, Code Quality & Security Analysis, Packaging & Versioning, Staging Deployment, and Production Deployment — with monitoring and notification hooks attached to every stage. Figure 1 illustrates the full workflow.

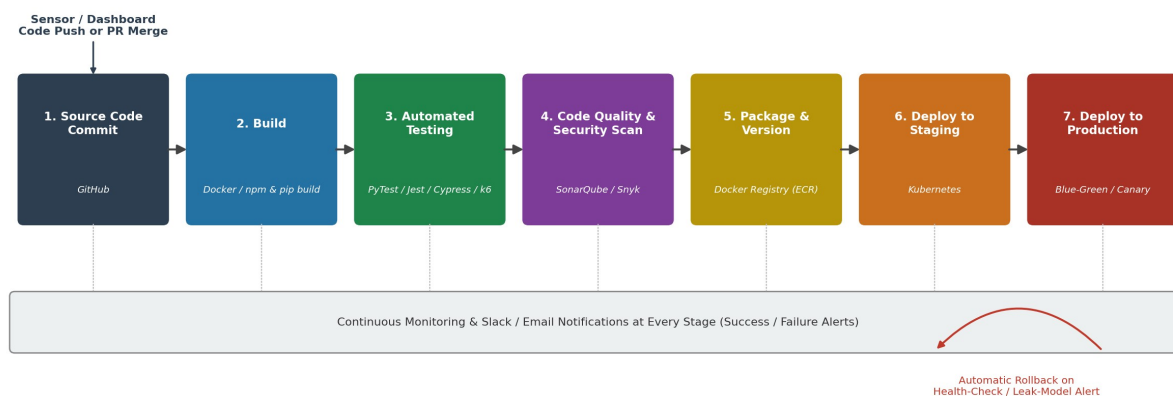


Figure 1: End-to-End CI/CD Pipeline Architecture for the Intelligent Water Distribution Monitoring and Leakage Detection Platform

2.2 Stage-by-Stage Workflow Explanation

- **Stage 1 — Source Code Management:** Developers commit to short-lived feature branches and open a Pull Request into develop. The PR automatically triggers the pipeline; direct pushes to main are blocked by branch-protection rules requiring at least one peer review and a passing pipeline run.
- **Stage 2 — Build:** The dashboard/GIS front end (React) is built with npm, the Node.js ingestion/API services are compiled, and the Python leak-detection model service is packaged with its dependencies; all services are containerized using multi-stage Docker builds to keep production images minimal.
- **Stage 3 — Automated Testing:** Unit, integration, and end-to-end test suites run in parallel containers, including dedicated tests that replay recorded sensor-telemetry streams (including known historical leak events) to validate the anomaly-detection logic.
- **Stage 4 — Code Quality & Security Scan:** Static code analysis (linting, complexity, code smells), Static Application Security Testing (SAST), and open-source dependency vulnerability scanning run against the codebase; a failing quality gate stops the pipeline before packaging.
- **Stage 5 — Package & Version:** A Docker image is built, tagged with the Git commit SHA and semantic version (e.g., v2.1.0); the leak-detection model artifact is separately versioned and registered in an MLflow model registry, signed, and pushed to a private container registry.
- **Stage 6 — Deploy to Staging:** The versioned image is deployed to the Kubernetes staging namespace automatically; smoke tests, Dynamic Application Security Testing (DAST), and a model shadow-test

(scoring live-replayed telemetry against both the current and candidate model without alerting) run against the live staging URL.

- **Stage 7 — Deploy to Production:** After a manual approval gate (QA/Release Manager sign-off), the image is promoted to production using a blue-green strategy with an initial canary traffic slice; full traffic is switched only after post-deployment health checks and alert-volume checks succeed, with automatic rollback on failure.

2.3 Branching Strategy Integrated with the Pipeline

The team follows a GitFlow-inspired branching model that maps directly onto the pipeline's trigger points, so that pipeline behaviour (which tests run, which environment is targeted) changes automatically with the branch type, as shown in Figure 2.

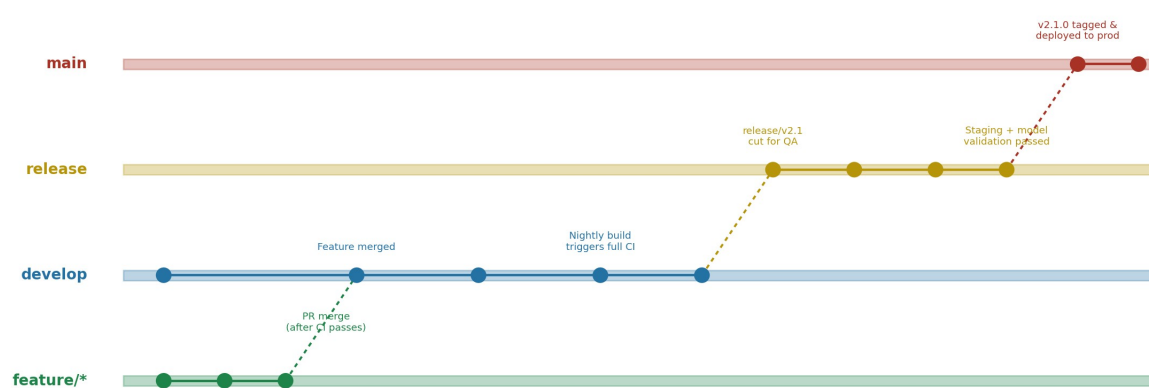


Figure 2: Git Branching Strategy (GitFlow) and its mapping to pipeline triggers

- **feature/*** — full unit + integration test run, deploys to a per-branch ephemeral preview environment for design/UX and model-experimentation review.
- **develop** — nightly full pipeline run including E2E, load tests, and a full model-accuracy regression run; auto-deployed to the Development environment.
- **release/*** — triggers the complete pipeline including security scans and model shadow-testing, and deploys to Staging for QA sign-off.
- **main** — triggers production deployment through the blue-green/canary strategy once release is approved and tagged.

2.4 Pipeline Triggers & Timing

Not every stage runs on every trigger — running the full pipeline (including load tests and full model regression) on each of dozens of daily commits would slow the team down without adding proportional safety. Trigger scope is therefore tuned per event type, as summarized below.

Trigger Event	Stages Executed	Typical Duration
Push to feature/*	Build, unit tests, integration tests, preview deploy	~4–6 minutes

Trigger Event	Stages Executed	Typical Duration
Pull request into develop	Build, unit + integration + E2E tests, code quality scan	~8–10 minutes
Nightly schedule on develop	Full suite including E2E, load tests, model regression, security scans, dev-environment deploy	~30–35 minutes
release/* branch cut	Full pipeline: all tests, SAST/DAST, model shadow-test, staging deployment	~40–45 minutes
Merge/tag on main	Package, production blue-green/canary deployment, health-check bake, notification	~10–15 minutes

3. Tools & Technology Selection

The tools below were selected to match the platform's technology stack (React front end, Node.js ingestion/API layer, Python ML model service, containerized microservices on Kubernetes) and its specific risk areas — high-throughput sensor ingestion, leak-model accuracy, and continuous availability.

Pipeline Stage	Selected Tool(s)	Justification
Source Code Management	GitHub + GitHub Actions	Native, tightly-integrated CI triggers on push/PR events; built-in branch-protection rules, code owners, and PR review gating fit the GitFlow model without extra tooling overhead.
Build	Docker (multi-stage builds), npm / Node.js and pip / Python build toolchains	Multi-stage Docker builds keep runtime images small and reproducible across dev, staging, and production, eliminating environment drift for the ingestion and model services.
Unit & Integration Testing	Jest (JS/TS), PyTest (model service), Supertest (API), Postman/Newman	Jest and PyTest are the de-facto standards for the JS and Python halves of the stack respectively, with fast parallel execution and coverage reporting; Supertest validates REST/MQTT-bridge API contracts.
End-to-End Testing	Cypress / Playwright	Simulates real operator/field-technician journeys — viewing the live map, receiving an alert, and closing a work order — essential for verifying the alerting and dispatch workflow end to end.
Sensor-Stream & Load Testing	k6, replayed MQTT telemetry fixtures	Scriptable, developer-friendly load testing that can simulate thousands of concurrent sensor connections and replay historical leak-event streams to validate real-time scalability and detection accuracy before release.
ML Model Tracking & Validation	MLflow	Versions leak-detection model artifacts, tracks precision/recall/false-positive-rate metrics per candidate model, and gates promotion on no regression versus the current production model.
Code Quality Analysis	SonarQube	Provides continuous static analysis, code-smell detection, maintainability ratings, and enforces a quality gate that blocks the pipeline on regressions.
Security Scanning (SAST/SCA)	Snyk, OWASP Dependency-Check	Detects vulnerable open-source dependencies and insecure coding patterns early, before they reach a container image — important given the platform's exposure to field devices and SCADA integration.
Dynamic Security Testing (DAST)	OWASP ZAP	Runs automated penetration-style scans against the live staging environment to catch runtime vulnerabilities (e.g., injection via device-reported metadata) missed by static analysis.
Containerization & Registry	Docker, Amazon ECR	Ensures identical, versioned artifacts are promoted across environments; ECR integrates natively with the chosen Kubernetes deployment target and supports image scanning.
Deployment Orchestration	Kubernetes (EKS), Helm	Kubernetes provides self-healing, horizontal auto-scaling for ingestion-heavy workloads and native support for blue-green/canary rollout strategies via Helm charts.

Pipeline Stage	Selected Tool(s)	Justification
Sensor Ingestion Broker	AWS IoT Core (MQTT) / EMQX	Managed, certificate-authenticated MQTT broker purpose-built for large fleets of field devices, reducing the operational burden of running a self-hosted broker.
Infrastructure as Code	Terraform	Version-controlled, repeatable provisioning of staging/production infrastructure, reducing configuration drift between environments.
Secrets Management	HashiCorp Vault / AWS Secrets Manager	Centralizes database credentials, device-certificate roots, and API keys outside of source control and pipeline logs.
Monitoring & Logging	Prometheus, Grafana, ELK Stack	Real-time dashboards and centralized logs are essential to catch ingestion lag, sensor dropout, or elevated false-alert rates immediately after each deployment.
Notifications	Slack, Email (via GitHub Actions/Webhook)	Immediate pipeline status alerts keep the small development team informed of failures without needing to actively monitor the CI dashboard.

3.1 Alternatives Considered

For several stages, credible alternative tools were evaluated before settling on the choices above; the reasoning behind each trade-off is summarized below so the selection is defensible rather than arbitrary.

- **CI orchestration — GitHub Actions vs. Jenkins:** Jenkins offers greater self-hosted flexibility but requires dedicated infrastructure and maintenance; for a small team, GitHub Actions' managed runners and native GitHub integration reduce operational overhead considerably.
- **Ingestion broker — AWS IoT Core vs. self-hosted EMQX:** A self-hosted EMQX cluster gives finer-grained control and avoids cloud lock-in, but AWS IoT Core's built-in device-certificate lifecycle management and native scaling reduce the security and operational burden for a small team; EMQX remains listed as a portable fallback.
- **Unit testing — Jest vs. Mocha:** Mocha/Chai is a viable alternative to Jest, but Jest's built-in snapshot testing, mocking, and coverage reporting reduce the number of additional libraries the team must configure and maintain.
- **E2E testing — Cypress vs. Selenium:** Selenium remains widely used, but Cypress's native browser execution and time-travel debugging significantly speed up authoring and diagnosing tests for the live-map and alert-workflow scenarios this platform requires.
- **Orchestration — Kubernetes vs. Docker Swarm:** Docker Swarm was considered for its simplicity, but Kubernetes' maturity, wide ecosystem support for blue-green/canary rollout patterns, and horizontal auto-scaling make it the stronger long-term fit as the sensor fleet grows.
- **ML tracking — MLflow vs. Weights & Biases:** Weights & Biases offers a richer hosted experiment-tracking UI, but MLflow's open-source, self-hostable model registry integrates more directly with the existing Docker/Kubernetes pipeline without an added external SaaS dependency for a data-sensitive utility client.

4. Automated Testing & Quality Integration

4.1 Testing Pyramid for the Platform

Testing is organized as a pyramid, weighted toward fast, cheap unit tests and narrowing toward slower, high-value end-to-end, load, and model-accuracy tests, all fully automated and wired into the pipeline so no release proceeds on a broken quality gate.

- **Unit Tests (Jest / PyTest):** Validate the sensor-data parsers, anomaly-scoring functions, leak-localization logic, and alert-routing rules in isolation. Target: greater than 85% statement coverage on the ingestion-validation and leak-scoring modules specifically, since these carry the highest safety and data-integrity risk.
- **Integration Tests (Supertest / Newman):** Validate REST/MQTT-bridge API contracts between field gateways, the ingestion service, the model-scoring service, and the persistence layer, including authentication/authorization checks for role-based permissions.
- **End-to-End Tests (Cypress):** Automate real user journeys: a simulated leak event appearing on the live map, an operator acknowledging and assigning it, a field technician closing the work order, and historical report export — run headlessly in CI on every release-branch build.
- **Load & Ingestion Tests (k6):** k6 scripts simulate thousands of concurrent sensor connections and bursty telemetry (e.g., a storm event causing a spike in pressure readings) against staging to confirm ingestion latency and alerting correctness stay within SLA before promotion to production.
- **Leak-Detection Model Validation:** Every candidate model is scored against a held-out, labelled dataset of historical leak and non-leak events; the pipeline blocks promotion unless precision, recall, and false-positive rate are at or above the current production model's benchmark.
- **Static Code Quality Gate:** SonarQube quality gate enforces thresholds on code coverage, duplication, cyclomatic complexity, and blocks merge on any new critical/blocker issue.
- **Security Testing (SAST/SCA):** Snyk and OWASP Dependency-Check scan every build for known CVEs in third-party packages (a real risk given the many npm and pip dependencies of a geospatial, real-time platform); builds fail on high/critical findings.
- **Dynamic Security Testing (DAST):** OWASP ZAP runs an automated scan against the staging deployment to catch injection, XSS, and authentication flaws before production release — particularly relevant given operator- and device-supplied metadata fields.

4.2 Pipeline Quality Gates

A build is only allowed to progress to the next stage if all gates at the current stage pass. This fail-fast design means expensive stages (staging/production deployment) never run on code that has already failed a cheaper, earlier check. Table 2 summarizes the pass/fail criteria enforced at each gate.

Gate	Pass Criteria
Unit/Integration Tests	100% of unit and integration tests pass; coverage on ingestion-validation/leak-scoring modules remains above 85%.
End-to-End Tests	All critical user-journey scenarios (leak alert, acknowledgement, dispatch, export) pass with no flaky retries exceeding threshold.
Leak-Model Validation	Candidate model's precision, recall, and false-positive rate on the held-out labelled

Gate	Pass Criteria
	dataset are at or above the current production model.
Code Quality (SonarQube)	Quality gate status is “Passed”: zero new blocker/critical issues, duplication below 3%.
Security Scan (Snyk/OWASP)	No unresolved high or critical severity vulnerabilities in dependencies or container base image.
DAST (OWASP ZAP)	No high-risk alerts on the staging deployment (e.g., injection, broken auth).
Production Health & Canary Check	Error rate, P95 latency, and alert-volume trend remain within SLA for the full post-deploy bake period, else automatic rollback triggers.

4.3 Test Environment & Test Data Management

Reliable automated testing depends as much on how test environments and data are managed as on the tests themselves. The pipeline follows these practices to keep test runs fast, isolated, and representative of production behaviour:

- **Ephemeral test infrastructure:** Each pipeline run spins up isolated, disposable test containers (database, MQTT broker, API, model service) via Docker Compose so parallel CI jobs never share state or interfere with one another.
- **Synthetic and anonymized telemetry:** Test suites run against synthetically generated sensor streams and anonymized replays of historical telemetry rather than live customer-utility data, eliminating any risk of exposing sensitive infrastructure data during CI runs.
- **Deterministic seeding:** Before each E2E run, the test database is seeded from a fixed, version-controlled fixture set (sample network topology, sensor inventory, and past incident history) so tests are deterministic and reproducible across runs.
- **Parallel test isolation:** Independent, parallel test workers each get their own database schema/namespace, cutting overall pipeline duration by running unit, integration, and E2E suites concurrently rather than sequentially.

Figure 3 below shows a representative excerpt of the pipeline's actual GitHub Actions configuration, tying together the build, test, quality-scan, packaging, and multi-environment deployment jobs described in this section and the next.

4.4 Continuous Feedback to Developers

Fast, visible feedback is what makes the quality gates above sustainable in daily practice rather than a source of friction. Two practices keep the loop tight:

- **Inline PR status checks:** Every pipeline run posts a status check directly on the pull request, with failing stages linked straight to their logs so a developer can diagnose a failure without leaving GitHub.
- **Annotated quality reports:** SonarQube, coverage, and model-metric reports are published as PR comments summarizing exactly which lines, files, or accuracy metrics caused a quality-gate failure, turning an abstract “build failed” into an actionable, line-level pointer.

```
name: Water Platform CI/CD Pipeline

on:
  push:
```

```

  branches: [ develop, release/*, main ]
pull_request:
  branches: [ develop ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install dependencies
        run: npm ci && pip install -r requirements.txt
      - name: Run unit tests (Jest + PyTest)
        run: npm run test:unit --coverage && pytest --cov=model_service
      - name: Run E2E tests (Cypress)
        run: npx cypress run --record
      - name: Validate leak-detection model
        run: python model/validate_model.py --min-recall 0.9
      - name: SonarQube code quality scan
        uses: sonarsource/sonarqube-scan-action@v2
      - name: Snyk dependency & container scan
        run: snyk test --severity-threshold=high

  build-and-push-image:
    needs: build-and-test
    steps:
      - name: Build Docker image
        run: docker build -t water-platform:${{ github.sha }} .
      - name: Push to ECR registry
        run: docker push $ECR_REGISTRY/water-platform:${{ github.sha }}

  deploy-staging:
    needs: build-and-push-image
    environment: staging
    steps:
      - name: Deploy to Kubernetes (staging)
        run: kubectl apply -f k8s/staging/ --record
      - name: Run smoke tests, DAST & model shadow-test

```

Figure 3: Representative GitHub Actions workflow configuration (ci-cd-pipeline.yml) implementing the build, test, quality-scan, package, and deploy jobs described above

5. Deployment, Security & Rollback Strategy

5.1 Multi-Environment Deployment Strategy

Code is promoted progressively through three environments, with automated gates and one manual approval before production, and a blue-green mechanism with canary traffic in production for zero-downtime releases, as shown in Figure 4. This progressive-promotion model exists because each environment answers a different question: Development confirms the code runs at all, Testing/QA confirms it behaves correctly against realistic multi-sensor scenarios, Staging confirms it is safe, secure, and accurate enough to expose to real infrastructure, and Production confirms it performs well under real sensor load — with each gate catching classes of defects that are cheaper to fix the earlier they are found.

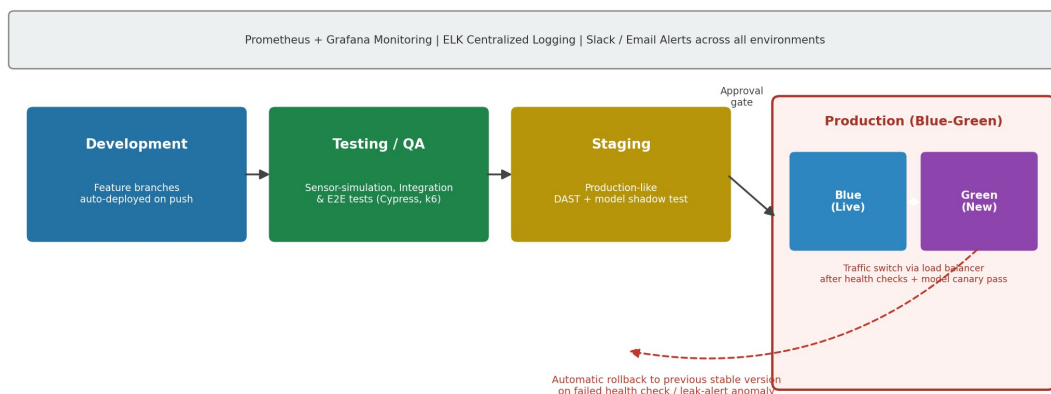


Figure 4: Multi-Environment Promotion Path with Blue-Green Production Deployment and Rollback

- **Development environment:** Every feature branch is auto-deployed to an ephemeral namespace for internal testing and design review; torn down automatically when the branch is merged or closed.
- **Testing/Staging environment:** A production-like environment used for full regression, load, security (DAST), and model shadow-testing; requires QA and data-science sign-off before promotion.
- **Production (blue-green with canary):** Runs two identical stacks — Blue (currently live) and Green (newly deployed). A small canary slice of live sensor traffic is routed to Green first; full traffic is switched at the load balancer only after the new version passes automated post-deployment health and alert-volume checks, giving true zero-downtime releases for continuous monitoring.

5.2 Security Practices Embedded in the Pipeline

- **1. Shift-left vulnerability scanning:** All source-to-image dependencies are scanned by Snyk/OWASP Dependency-Check on every build; container images are additionally scanned with Trivy before being pushed to the registry.
- **2. Secrets management:** Database credentials, JWT signing keys, MQTT device-certificate roots, and third-party API keys are injected at deploy time from Vault/AWS Secrets Manager — never stored in the repository or pipeline YAML.

- **3. Device and transport authentication:** Field sensors and gateways authenticate to the ingestion broker using per-device X.509 certificates over TLS 1.3; the pipeline includes a certificate-expiry check as part of the pre-deployment gate.
- **4. Role-based deployment access:** Deployment credentials for staging and production are scoped separately, and production deploys require an explicit manual approval step from a release manager, satisfying least-privilege and separation-of-duties principles.
- **5. Runtime security testing (DAST):** OWASP ZAP actively probes the staging build for injection and authentication vulnerabilities — a direct mitigation for the operator- and device-facing interfaces this platform exposes.

5.3 Monitoring, Alerting & Rollback

Prometheus scrapes application and infrastructure metrics (ingestion lag, sensor dropout rate, API latency, leak-alert volume and rate-of-change) with Grafana dashboards for visualization, while the ELK stack centralizes logs from all microservices for fast root-cause diagnosis. A post-deployment automated health check monitors error rate, latency, and alert-volume trend for a defined bake period (for example, 15 minutes) immediately after the blue-green traffic switch; if error thresholds are breached or the alert-volume trend deviates sharply from the historical baseline (suggesting the new model or code is over- or under-alerting), the pipeline automatically reverts the load balancer to the previous stable (blue) stack without requiring human intervention, and a rollback notification is sent immediately.

All pipeline outcomes — success, failure, or rollback — trigger real-time Slack and email notifications to the development and release-management channels, ensuring the team can react within minutes even outside working hours, since a missed leak has real-world consequences around the clock.

5.4 Rollback Procedure — Step by Step

- **1. Detect:** Prometheus alert rules continuously evaluate error rate, P95 latency, sensor-dropout rate, and leak-alert volume against the production “green” stack for the full post-deploy bake window.
- **2. Alert:** If any threshold is breached, an automated Grafana/Alertmanager webhook fires, notifying the on-call engineer and simultaneously invoking the rollback pipeline job.
- **3. Revert traffic:** The load balancer's target group is switched back to the previously live “blue” stack in seconds, since blue is left running (not torn down) until the bake period completes successfully.
- **4. Preserve evidence:** The failed “green” deployment is left running in isolation (not destroyed) so engineers can pull logs and reproduce the failure without impacting live monitoring.
- **5. Notify:** A Slack message and email summarize what was rolled back, the triggering metric, and a link to the relevant Grafana dashboard and ELK log query.

6. Pipeline Execution Walkthrough & Documentation

6.1 End-to-End Walkthrough: Commit to Deployment

This section documents a single, concrete pipeline run to illustrate how the architecture in Section 2 behaves in practice, for a release-branch build.

- **1. Trigger:** A developer merges a pull request implementing an improved pressure-drop anomaly threshold into release/v2.1; GitHub Actions detects the push and starts Run #217.
- **2. Checkout & Build:** The commit (a41f7c2) is checked out in 4 seconds, dependencies installed via npm ci and pip install in ~52 seconds.
- **3. Testing:** 246 Jest/PyTest unit tests pass with 88.1% coverage on ingestion and leak-scoring modules; 36 Cypress end-to-end scenarios covering live-map alerts, acknowledgement, and dispatch all pass.
- **4. Model Validation:** The candidate model scores 0.93 recall and 0.04 false-positive rate on the held-out leak-event dataset, matching or exceeding the current production benchmark (0.91 recall, 0.05 false-positive rate).
- **5. Quality & Security:** SonarQube reports a passing quality gate with zero blocker/critical issues; Snyk finds no high/critical dependency vulnerabilities.
- **6. Packaging:** The Docker image water-platform:a41f7c2 is built and pushed to the private ECR registry, and the validated model artifact is registered in MLflow as v2.1-candidate.
- **7. Staging Deployment:** The image is deployed to the staging Kubernetes namespace; smoke tests and a live shadow-scoring run confirm the service is healthy and the model behaves as expected against real (replayed) telemetry.
- **8. Production Deployment:** After sign-off, a 10% canary traffic slice is routed to the new stack for 15 minutes; health and alert-volume checks pass, the blue-green switch completes, and a Slack notification confirms the successful release to the #deployments channel.

6.2 Traceability Between Requirements and Pipeline Design

Identified Requirement	Pipeline Feature Addressing It
Data integrity of sensor ingestion	High unit-test coverage requirement on ingestion-validation modules; SonarQube quality gate blocks regressions.
Accuracy of leak-detection alerts	Automated model-validation gate comparing candidate precision/recall/false-positive rate against the current production benchmark before every release.
Zero-downtime releases	Blue-green production deployment with canary traffic and automated health-check gating before full traffic switch.
Security & privacy of infrastructure data	SAST (Snyk/OWASP Dependency-Check) and DAST (OWASP ZAP) integrated at build and staging stages; device certificates and secrets never stored in code.
Fast recovery from bad releases	Automated rollback to the previous stable blue-green stack on failed health or alert-volume checks, with instant Slack/email alerts.

6.3 Key Pipeline Metrics Tracked

To continuously improve the pipeline itself, the team tracks a small set of DevOps performance indicators (aligned with common DORA metrics) on a rolling basis, reviewed at each sprint retrospective:

- **Pipeline duration:** Average of 7 minutes for feature-branch feedback and roughly 13 minutes for a full production deployment run, keeping developer feedback loops short.
- **Test flakiness rate:** Tracked separately per stage (unit, E2E, model validation) to flag flaky tests early; target below 2% flake rate for E2E suites.
- **Deployment frequency:** Multiple production deployments per week are supported safely due to the automated gates, canary rollout, and blue-green rollback safety net.
- **Change failure rate:** Percentage of production deployments that trigger an automatic rollback; sustained target below 5%, investigated whenever exceeded.
- **Mean time to recovery (MTTR):** Time from a rollback-triggering alert to full traffic restoration on the previous stable version; target under 2 minutes given the automated blue-green switch.
- **Model drift indicator:** Weekly comparison of production leak-alert precision/recall against the validation benchmark, flagging seasonal drift (e.g., changing usage patterns) before it degrades detection quality.

7. Risk Assessment & Mitigation

The table below identifies the principal risks specific to a real-time, safety-relevant, sensor-driven monitoring platform and the corresponding pipeline-level mitigation designed into the CI/CD workflow described in the previous sections.

Risk	Impact	Pipeline Mitigation
Faulty leak-detection model misses or falsely flags leaks	High — water loss, property damage, alert fatigue	Mandatory model-validation gate comparing precision/recall/false-positive rate against the current production benchmark; dedicated replay tests using historical leak-event data.
Deployment interrupts continuous live monitoring	High — operators blind to real leaks during the gap	Blue-green deployment with canary traffic; full traffic only switches after health and alert-volume checks pass.
Sensor telemetry burst overwhelms ingestion during storm events	Medium–High — dropped readings, delayed alerts	k6 load tests simulating bursty, high-volume telemetry gate promotion from staging to production; ingestion auto-scales horizontally on Kubernetes.
Vulnerable dependency exposes utility infrastructure data	High — data breach, compliance exposure	Snyk/OWASP Dependency-Check gate blocks build on high/critical CVEs; scheduled nightly re-scan of production images.
Spoofed or tampered sensor telemetry	High — false alerts or masked real leaks	Per-device X.509 certificate authentication over TLS to the ingestion broker; anomalous device-identity patterns flagged by monitoring.
Bad release ships to the whole network simultaneously	High — widespread monitoring outage	Blue-green with canary and automated health-check rollback limits blast radius and enables sub-minute recovery.
Secrets leaked via repository or CI logs	High — credential compromise	Vault/AWS Secrets Manager injects secrets at runtime only; pipeline configured to mask secret values in logs.
Model drift as seasonal water-usage patterns change	Medium — gradually degraded accuracy	Weekly production accuracy comparison against benchmark; retraining and re-validation triggered when drift exceeds threshold.

8. Ethical & Compliance Considerations

CO4 explicitly emphasizes ethics alongside security and reliability. For a platform that monitors public water infrastructure and informs field dispatch decisions, the CI/CD pipeline was designed with the following ethical and compliance commitments built into its automated checks rather than treated as manual afterthoughts:

- **Duty of care for public infrastructure and safety:** Because a missed or delayed leak alert can lead to water loss, property damage, or road/sinkhole hazards, the pipeline treats any regression in leak-detection accuracy as a release-blocking defect, not a lower-priority bug — reflecting an ethical obligation to the communities the utility serves.
- **Avoiding alert fatigue:** The false-positive-rate gate on every candidate model exists specifically to protect field technicians from being desensitized by excessive false alarms, which is itself a safety risk if it causes a genuine alert to be deprioritized.
- **Data minimization and access control:** Every stage of the pipeline, and every environment, enforces role-based access so that only authorized services and personnel can read production telemetry and consumption data; deployment to production additionally requires named human approval, creating an auditable accountability trail.
- **Privacy of consumption data:** Test suites use synthetic and anonymized telemetry rather than real customer/utility data (see Section 4.3), so engineers and automated tooling never gain incidental access to household or facility water-usage patterns during development or CI runs.
- **Auditability:** Full audit logs of every pipeline run (who triggered it, what changed, what was deployed) are retained in the CI system and centralized logging stack, supporting transparent post-incident review and satisfying common data-protection and utility-regulator audit requirements.
- **Equitable monitoring coverage:** Load and scalability tests validate that ingestion and alerting performance is consistent across the full sensor fleet rather than only the busiest zones, so service reliability does not inadvertently favor higher-value districts over others.
- **Preventing misuse of the platform:** Dependency scanning, device-certificate authentication, and DAST checks are run on every release specifically to prevent the platform from being used as a vector to compromise SCADA-adjacent systems or spoof infrastructure data, in line with a duty of care for critical infrastructure.

9. Conclusion

The CI/CD pipeline designed for the Intelligent Water Distribution Monitoring and Leakage Detection Platform automates the full journey from source-code commit to production deployment across seven well-defined stages, using a GitFlow-based branching model to control what level of testing and which environment each change reaches. The design directly addresses the platform's defining risks — accuracy of the leak-detection model, reliability of high-throughput sensor ingestion, and the need for zero-downtime monitoring — through a layered testing pyramid, mandatory code-quality, security, and model-accuracy gates (SAST, DAST, dependency scanning, precision/recall validation), a blue-green production strategy with canary traffic and automated health-check-driven rollback, and continuous monitoring with instant team notifications. Together these practices ensure that new features and model improvements reach operators quickly, without ever compromising the reliability of live leak monitoring, fulfilling the CO4 objective of implementing robust testing, quality assurance, and DevOps practices with an emphasis on security, reliability, and ethics.

Looking ahead, the pipeline is designed to evolve alongside the platform: progressive canary percentages could be layered on top of the existing blue-green strategy for even finer-grained model rollout control, chaos-engineering drills could be scheduled against staging to validate resilience of the ingestion pipeline under induced sensor-network failure, and the current manual production-approval gate could be progressively automated as confidence in the automated quality and model-accuracy gates grows — without weakening the security, reliability, and ethical safeguards documented in this design.

Appendix A: Glossary of Key Terms

Term	Definition
CI/CD	Continuous Integration / Continuous Deployment — practices that automatically build, test, and release code changes frequently and reliably.
MQTT	A lightweight publish-subscribe messaging protocol widely used for IoT sensor telemetry over constrained networks.
SCADA	Supervisory Control and Data Acquisition — industrial control systems used by utilities to monitor and control field infrastructure.
Non-Revenue Water (NRW)	Water that has been produced but is lost before reaching the customer, largely through leaks; a key metric leak-detection platforms aim to reduce.
Blue-Green Deployment	A release strategy running two identical production environments (Blue and Green); traffic is switched to the newly deployed version only after it passes health checks, enabling zero-downtime releases and instant rollback.
Canary Release	A deployment technique that gradually shifts a small percentage of production traffic to a new version before a full rollout, limiting the impact of defects.
SAST	Static Application Security Testing — analyzing source code for security vulnerabilities without executing the program.
DAST	Dynamic Application Security Testing — probing a running application for vulnerabilities by simulating attacks (e.g., OWASP ZAP).
SCA	Software Composition Analysis — scanning third-party/open-source dependencies for known vulnerabilities (e.g., Snyk).
Quality Gate	A defined set of pass/fail conditions (test coverage, code smells, vulnerabilities, model accuracy) that a build must satisfy before proceeding to the next pipeline stage.
Model Shadow-Testing	Running a candidate ML model alongside the live production model on real traffic without acting on its output, to compare accuracy before promotion.
Rollback	Automatically or manually reverting a deployment to the last known-good version in response to a failure.
Infrastructure as Code (IaC)	Managing and provisioning infrastructure (servers, networks) through version-controlled configuration files (e.g., Terraform) instead of manual processes.
GitFlow	A branching model that uses dedicated long-lived branches (main, develop) and short-lived supporting branches (feature, release, hotfix) to structure a project's development history.
Smoke Test	A quick, shallow test run immediately after deployment to confirm the application starts and its core functions work before deeper testing proceeds.
Bake Period	A defined observation window after a deployment during which health metrics are monitored before the release is considered fully successful.
MTTR	Mean Time to Recovery — the average time taken to restore service after a failure is detected.
DORA Metrics	A set of four key software-delivery performance indicators (deployment frequency, lead time for changes, change failure rate, and time to restore service) defined by the DevOps Research and Assessment team.

Appendix B: Roles & Responsibilities (RACI)

The table below clarifies who is Responsible, Accountable, Consulted, or Informed at each major pipeline stage, ensuring clear ownership across the development, data-science, QA, and release-management functions. R = Responsible (does the work), A = Accountable (owns the outcome), C = Consulted (provides input), I = Informed (kept up to date).

Pipeline Activity	Developer	Data Scientist	QA / Reviewer	Release Manager	DevOps/SRE
Write & commit code	R, A	C	C	I	I
Train & validate leak model	C	R, A	C	I	I
Review pull request	C	C	R, A	I	I
Maintain pipeline (YAML/scripts)	C	I	I	I	R, A
Sign off staging build	I	C	R, A	C	I
Approve production release	I	C	C	R, A	I
Monitor post-deploy health	I	C	I	C	R, A
Execute rollback	I	I	I	A	R
Manage secrets & credentials	I	I	I	C	R, A

Document Control

Field	Detail
Document Title	CI/CD Pipeline Design — Intelligent Water Distribution Monitoring and Leakage Detection Platform
Prepared For	CO4 Assessment Tool 2 — CI/CD Pipeline Design Exercise, CSA10 / CSA1063 Software Engineering
Version	1.0